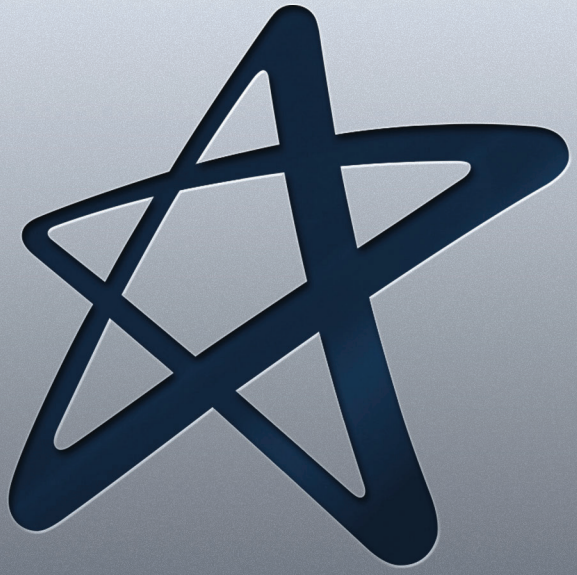
Programação para Dispositivos Móveis



Cruzeiro do Sul Virtual
Educação a Distância

Material Teórico

Menus e Banco de Dados

Responsável pelo Conteúdo:
Prof. Ms. Luis Carlos Reis

Revisão Textual:
Prof. Ms. Douglas Almendro

Revisão Textual:
Profa. Ms. Luciene Oliveira da Costa Santos

UNIDADE Menus e Banco de Dados

- Introdução
- Trabalhando com Menus em Android
- Banco de Dados
- Objeto DAO



OBJETIVO DE APRENDIZADO

- Demonstrar como a aplicação pode oferecer as opções aos usuários por meio de menu, o qual será exibido em 3 formas.
- Trabalhar com o armazenamento de informações, ou seja, programaremos Android juntamente com banco de dados.

ORIENTAÇÕES

Com o conhecimento adquirido nos conceitos iniciais, iremos trabalhar com menus e submenus e, também, com armazenamento de informações.

Para obter um bom aproveitamento nesta unidade, vamos conferir sua estrutura:

Conteúdo Teórico: neste link, você encontrará o material principal de estudos na forma de texto escrito.

Atividade de Sistematização: os exercícios disponibilizados são de autocorreção e visam à pratica do que aprendeu na disciplina e à identificação dos pontos que precisa prestar mais atenção, ou pedir esclarecimentos a seu tutor. Além disso, as notas atribuídas aos exercícios serão parte de sua média final na disciplina.

Atividade de Aprofundamento: ver a atividade que será disponibilizada.

Material Complementar e Referências Bibliográficas: nestes links, você poderá ampliar seus conhecimentos.

Uma última recomendação: caso tenha problemas para acessar algum item da disciplina, ou dúvidas em relação ao conteúdo, acione o botão mensagens e entre em contato com seu professor-tutor.

Contextualização

A utilização de menu e submenu é uma parte muito importante no desenvolvimento, pois os menus são utilizados como atalhos rápidos para acessos às principais funcionalidades dos sistemas.

Para armazenamento das informações, é essencial um lugar para guardarmos os dados. Em cada dispositivo, é possível armazená-los num banco de dados.

Nesta unidade, veremos a criação de menu e a utilização de banco de dados.

Introdução

Nesta aula, iremos aprender a fazer um menu simples para seu aplicativo Android. Existem 3 tipos de Menus para Android:

Menu de opções e barra de ação, onde ficam as ações que tem impacto sobre o aplicativo, como por exemplo: mandar e-mails e configurações.

Menu de contexto e ação contextual, que aparece quando o usuário realiza um clique longo, chamado de menu flutuante.

Por último, o menu pop-up que oferece uma lista vertical de itens que está ancorado a visão que invocou o menu.

Posteriormente, finalizaremos a aula com a utilização de banco de dados, pois todos nós conhecemos a importância da utilização de bancos de dados (ou arquivos, caso não exista a possibilidade de processamento de BDs) para armazenar dados em forma permanente, e o Android fornece um mecanismo para criar BDs dentro da aplicação, acessar e modificar seus dados. Para estudar os dois conceitos, iremos finalizar desenvolvendo um projeto para cadastrar alunos de uma instituição de ensino.

Trabalhando com Menus em Android

Existem duas formas para se trabalhar com menus:

- Criação de Menus via XML.
- Criação de Menus via codificação Java.

O resultado obtido é o mesmo, só mudando a forma de programar. Primeiramente, iremos abordar o tema utilizando em XML e, posteriormente, utilizando a codificação Java.

Podemos trabalhar com vários tipos de Menus, como: Menus de Contextos, Menus de Opções e Submenus. Nesta aula, sobrescrevemos alguns métodos da classe Activity utilizando o "Override", que no português significa Sobrescrita:

Tabela 1

Método	Descrição
public boolean onCreateOptionsMenu(menumenu)	Método booleano responsável pela criação dos itens do Menu, Sub-Menu ou Menus de Contexto
public boolean onOptionsItemSelected(menuItemitem)	Método booleano responsável por chamar as rotinas dos itens dos menus

Criação de Menus Via XML

O primeiro passo seria criar um Projeto Android clicando em "File/New/Android Project..." definindo o nome do projeto como "**Menu_Xml**".

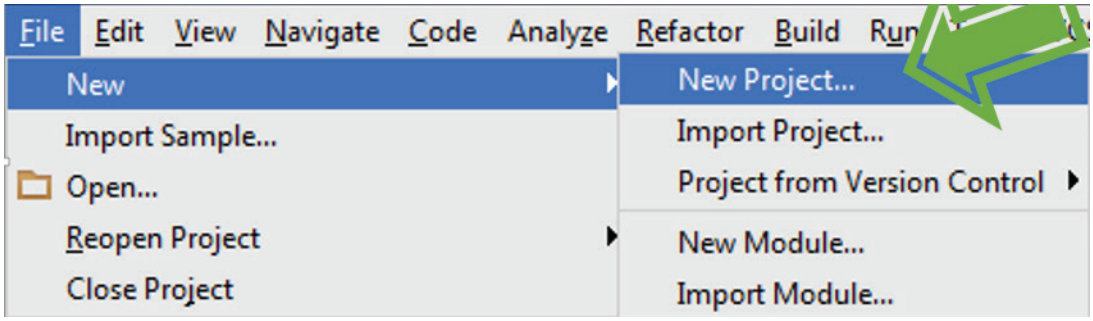


Figura 1

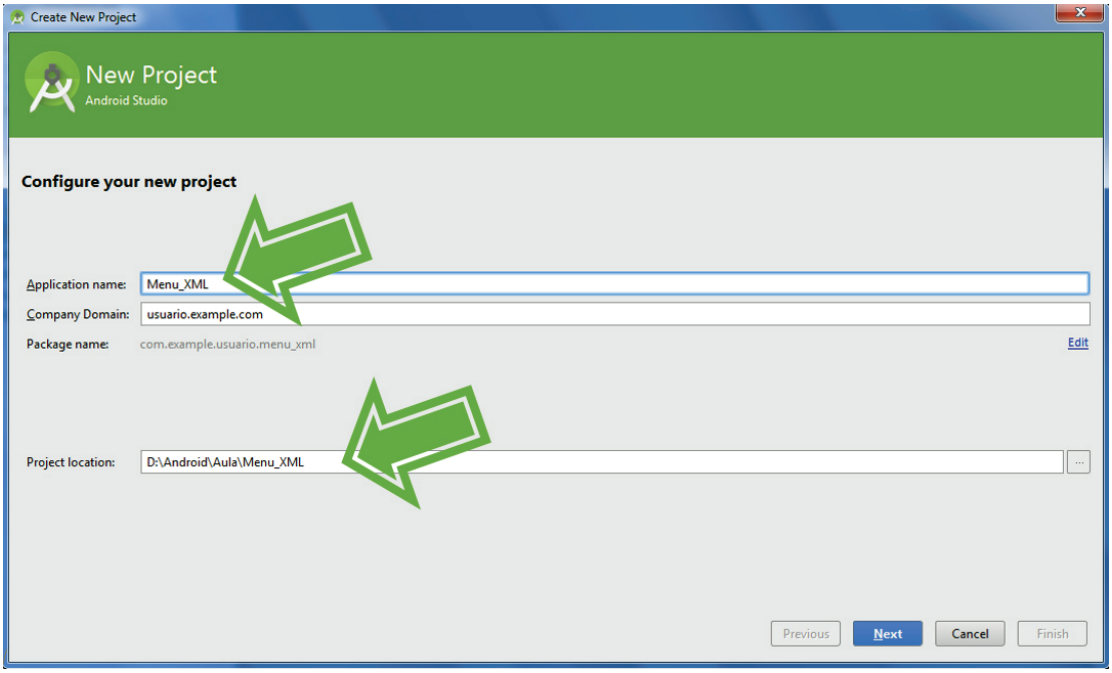


Figura 2

Localize o diretório "**res\menu**".

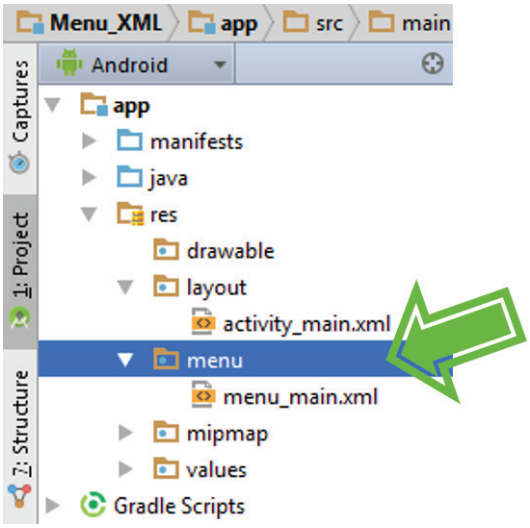


Figura 3

Dentro dessa pasta, criamos o arquivo XML responsável pela criação do Menu. Para isso, clique com o botão direito escolhendo "New/Menu resource file" e definindo o nome para "**lista.xml**".

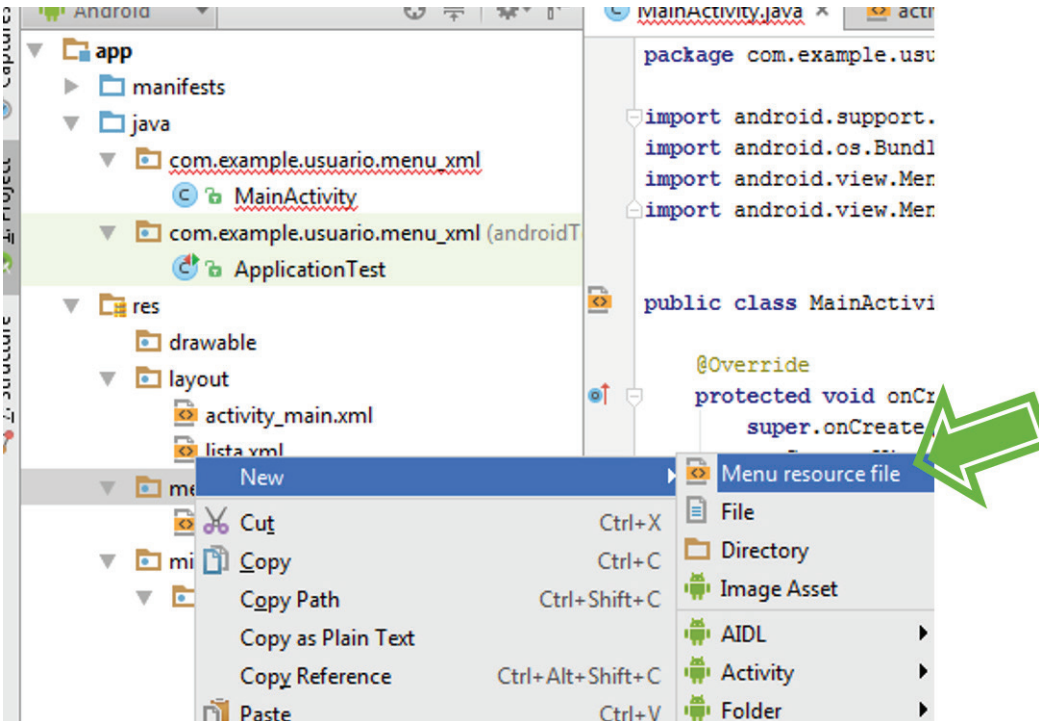


Figura 4

Caso queira, você poderá adicionar imagens no diretório "drawable" seguindo praticamente os mesmos passos descritos anteriormente. Estas imagens poderão servir de ícones para o Menu quando referenciadas pela tab: android:icon="@drawable/imgem01".

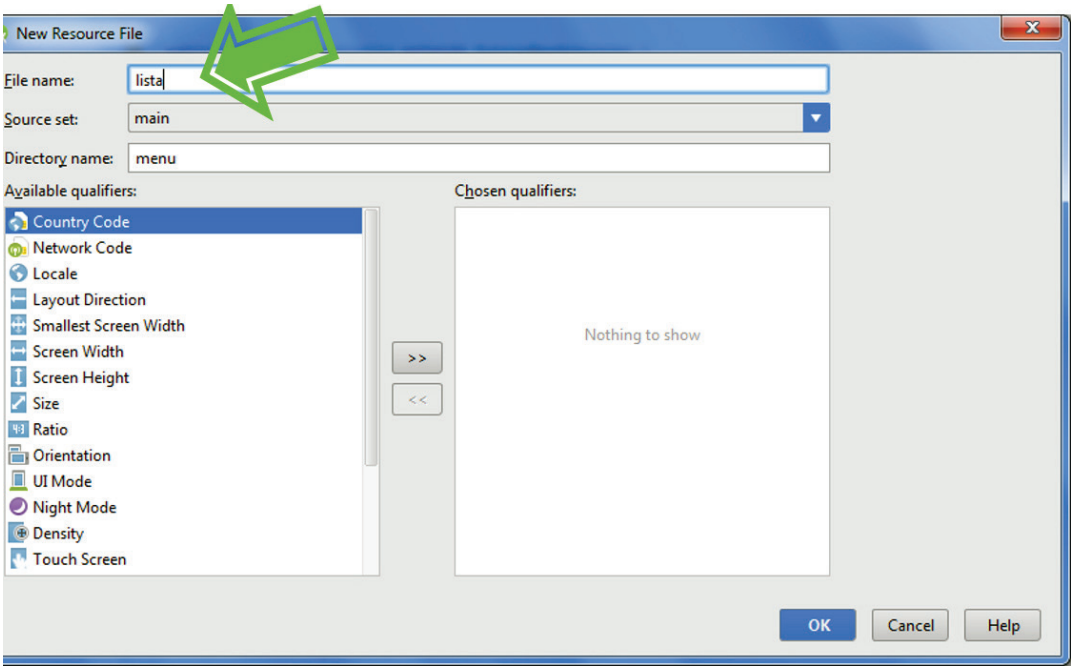


Figura 5

No arquivo lista.xml, iremos definir o id e o nome para ser utilizada no menu. Segue adiante a codificação:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/an-
droid">
 <item
 android:id="@+id/Cadastrar"
 android:title="Cadastrar"/>
 <item
 android:id="@+id/Movimentos"
 android:title="Movimentos"/>
 <item
 android:id="@+id/Caixa"
 android:title="Caixa"/>
</menu>
```

É muito simples, utilizamos apenas duas TAGS: a primeira – o Menu; e a segunda – os Itens.

Localize a classe "MainActivity" para implementarmos os métodos descritos acima. Importe o pacote AlertDialog, pois utilizaremos posteriormente:

```
import android.app.AlertDialog;
```

Ficará desta forma:

```
import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.app.AlertDialog;
```

No método onCreateOptionsMenu(), sobrescreva os métodos abaixo:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
 // Inflate the menu; this adds items to the action bar if it
is present.
 getMenuInflater().inflate(R.menu.Lista, menu);
 return true;
}
```

A método "inflate" será o responsável por capturar todos os itens do menu localizado no arquivo lista.xml criado anteriormente.

O método booleano OnOptionsItemSelected() é responsável por tratar os itens pela sua identificação "getItemId". Este método é responsável por chamar as rotinas necessárias ao utilizar os itens de Menu e deverá ficar da seguinte forma:

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
 int id = item.getItemId();

 //Realizar um case pelo Id dos itens e logo em seguida mos-
trar uma mensagem ao usuario
 switch (item.getItemId())
 {
 case R.id.Cadastrar: Mensagem("Cadastrar", "Cadastro!");
  break;
 case R.id.Movimentos: Mensagem("Movimentos", "Movimentos!");
  break;
 case R.id.Caixa: Mensagem("Caixa", "Caixa!");
  break;
 }
 //Retornar a classe pai
 return super.onOptionsItemSelected(item);

}

//Rotina para exibir mensagem ao usuario
public void Mensagem(String titulo, String texto)
{
 AlertDialog.Builder mensagem = new AlertDialog.
Builder(MainActivity.this);
 mensagem.setTitle(titulo);
 mensagem.setMessage(texto);
 mensagem.setNeutralButton("OK", null);
 mensagem.show();
}
```

A qualquer momento, você poderá "recompilar" seu projeto para verificar se existem erros de programação.

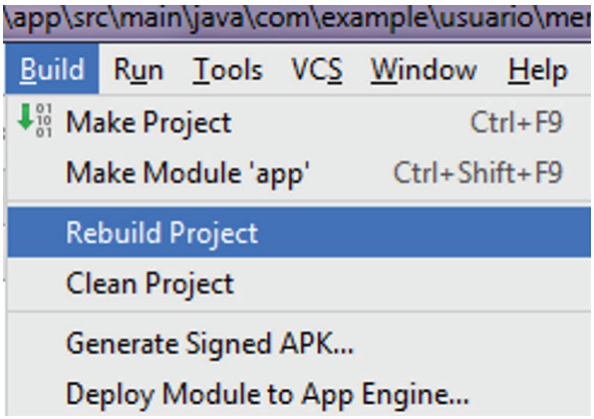
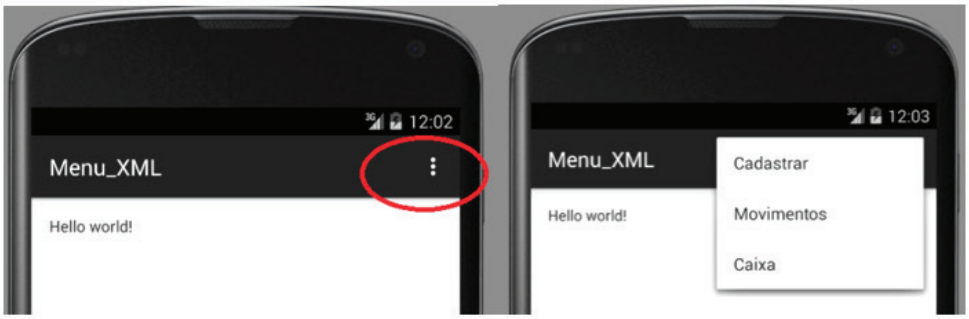


Figura 6

Execute o projeto e selecionando o menu abaixo. O mesmo deverá ficar idêntico à figura abaixo:



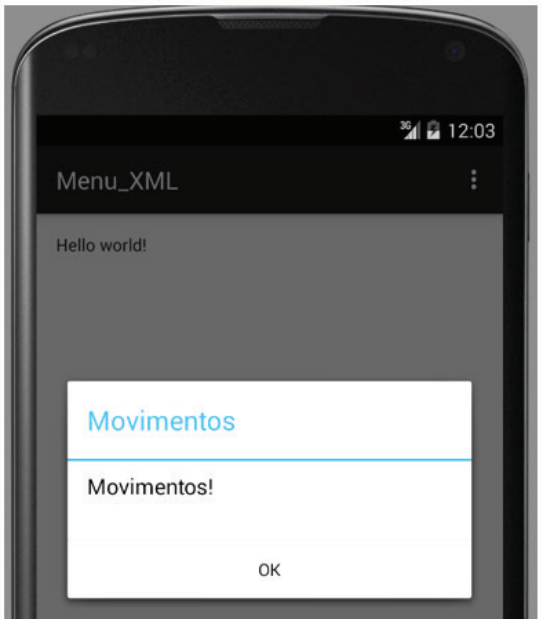


Figura 7

Criação de Menus via programação Java

A criação de Menus via programação é parecida com via XML, a única diferença é que não precisaremos do arquivo XML, ou seja, programaremos todo o código. Para isso, crie outro Projeto Android, clicando em "File/New/New Project...", definindo o nome como "Menu_Java". Localize o diretório "src" para implementarmos em nossa "Activity" o código necessário. Faremos um pouco diferente neste exemplo, programaremos a interface "Sub menus". Veja o código a seguir:

Importe os pacotes AlertDialog e SubMenu:

```
import android.view.SubMenu;
    import android.app.AlertDialog;



Ficará desta forma:
import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.SubMenu;
import android.app.AlertDialog;

No método onCreateOptionsMenu(), sobrescreva os métodos
abaixo:
@Override
public boolean onCreateOptionsMenu(Menu menu) {
 boolean resultado = super.onCreateOptionsMenu(menu);
 menu.add(0,1,0,"Cadastros");
 menu.add(0,2,0,"Movimentos");
 menu.add(0,3,0,"Caixa");

 SubMenu utilitario = menu.addSubMenu("Relatorios");
 utilitario.add(0,4,0,"Contas a Pagar");
 utilitario.add(0,5,0,"Contas a Receber");

 return resultado;
}
```

Neste caso, não utilizamos o método "Inflater", pois estamos criando o menu e os itens do submenu via programação. O objeto "menu" implementa o método "add", tendo como parâmetros: o grupo, o item, a ordenação e o título.

O método booleano OnOptionsItemSelected() segue a mesma lógica no que foi criado anteriormente, sendo responsável por tratar os itens pela sua identificação "getItemId", chamando as rotinas necessárias ao utilizar os itens de Menu. A única diferença é que possuímos alguns itens a mais.

A classe Builder novamente está presente em nosso exemplo, tendo como papel mostrar uma mensagem informativa ao usuário.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
 int id = item.getItemId();

 switch (item.getItemId())
 {
 case 1: Mensagem("Cadastros", "Cadastro!");
  break;
 case 2: Mensagem("Movimentos", "Movimentos!");
  break;
 case 3: Mensagem("Caixa", "Caixa!");
  break;
 case 4: Mensagem("Contas a Pagar", "Contas a Pagar!");
  break;
 case 5: Mensagem("Contas a Receber", "Contas a Receber!");
  break;
 }
 //retornar a classe pai
 return super.onOptionsItemSelected(item);

}

//Rotina para exibir mensagem ao usuario
public void Mensagem(String titulo, String texto)
{
 AlertDialog.Builder mensagem = new AlertDialog.
Builder(MainActivity.this);
 mensagem.setTitle(titulo);
 mensagem.setMessage(texto);
 mensagem.setNeutralButton("OK", null);
 mensagem.show();
}
```

O exemplo deverá ficar idêntico à figura abaixo:

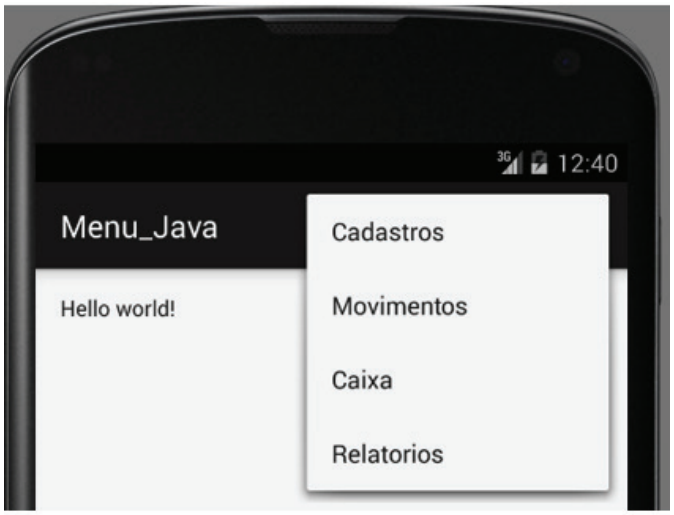


Figura 8

No Menu Relatórios, teremos mais 2 Sub Menus, conforme figura abaixo:



Figura 9

Observações:

Os métodos **onCreateOptionsMenu** e **onOptionsItemSelected** devem estar dentro da classe derivada de Activity, que define uma tela da aplicação. Em um programa com várias telas, cada uma poderá ter seu próprio menu e submenus.

Os métodos *onCreateOptionsMenu* e *onOptionsItemSelected* se comportam como eventos e serão chamados pelo sistema operacional do aparelho no momento adequado.

Observe a utilização das classes *Menu* e *SubMenu na* lógica dentro do método *onCreateOptionsMenu*.

O menu será ativado quando seja pressionada a opção **Menu** do celular ou dispositivo Android (poderá ser uma tecla física).

Também é possível modificar os menus durante a execução do programa. Estas duas últimas possibilidades não foram apresentadas nesta aula, mas fica como sugestão para aprofundar como estudo individual.

Banco de Dados

Acessando Banco de Dados em Android

De nada adianta fazermos um formulário para adicionar dados na tela e não termos onde guardá-los. Será necessário persisti-los de alguma forma. Apesar da grande e crescente capacidade dos aparelhos móveis, em particular aqueles utilizando Android, ainda não é viável guardar um banco de dados completo nesses dispositivos.

Para persistirmos os dados no Android, usaremos o SQLLite, que seria uma biblioteca de bancos de dados baseada em SQL e que atua como um "pequeno" SGBD e alguns arquivos de sistema, os Shared-Preferences, que nos permitem acessar dados persistidos.



Conheça mais sobre o SQLLite no link a seguir: www.sqlite.org

SQLite é o banco de dados padrão em várias outras aplicações: Firefox, iPhone, Symbia, Skype, etc. SQLite não possui licença, sendo de domínio público. O Android SDK oferece uma biblioteca de classes para manipular BDs de tipo SQLite. Dentro do pacote android.database.sqlite, encontraremos as classes SQLiteDatabase e SQLiteOpenHelper, que utilizaremos nas descrições nesta aula.

Seguindo a ideia do padrão MVC, dividiremos a nossa aplicação em três camadas. A primeira será a de visualização, no qual iremos trabalhar com o layout, a segunda será a Controller, que irá controlar a integração entre nossa View e as outras classes através dos listeners.

Podemos manipular transações se for necessário (métodos beginTransaction, endTransaction, setTransactionSuccessful).

Android sugere a utilização da classe SQLiteOpenHelper para facilitar o mecanismo de criação/abertura do Banco de Dados e as alterações que sejam efetuadas. Basicamente, criaremos uma classe (**AlunoDAO.java**) derivada de SQLiteOpenHelper, onde especificaremos o nome do BD e a estrutura da tabela do BD.

Primeiramente iremos criar a parte gráfica, através da camada View, onde definiremos o menu e o respectivo layout.

Iremos desenvolver um projeto para cadastrar alunos de uma universidade onde irá registrar alunos e suas respectivas notas. Crie um projeto com nome **CadastroAlunos**, e iremos dar o nome de Alunos para a **Activity**.

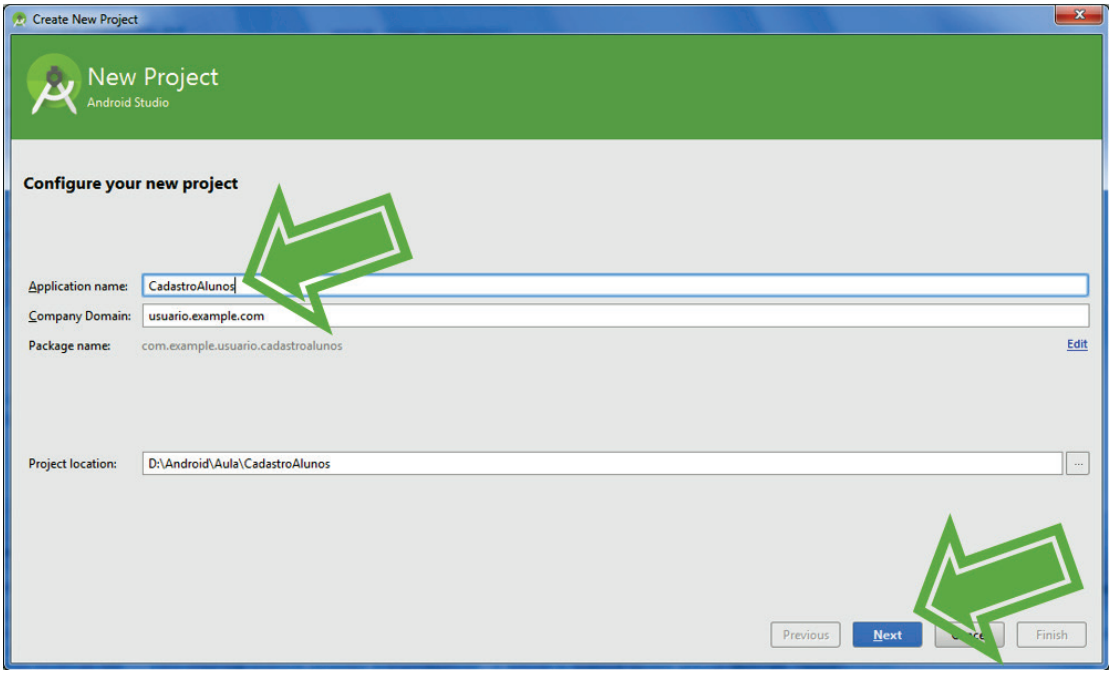


Figura 10

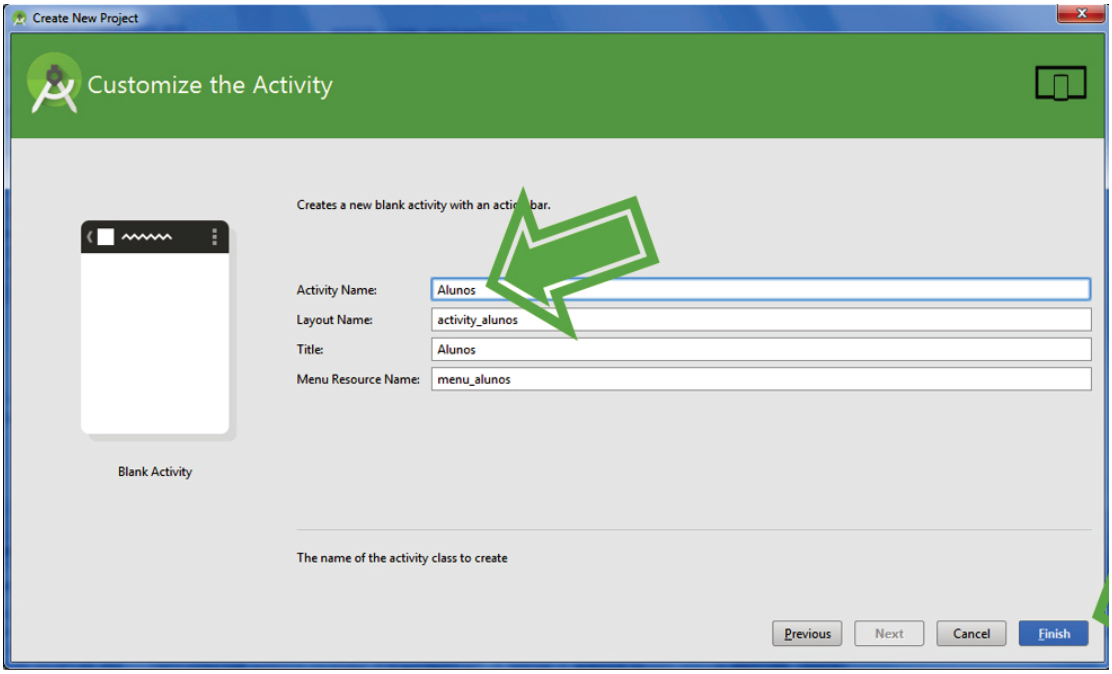


Figura 11

Para podermos exibir os alunos, iremos modificar o layout activity_alunos.xml que fica em /res/layout.

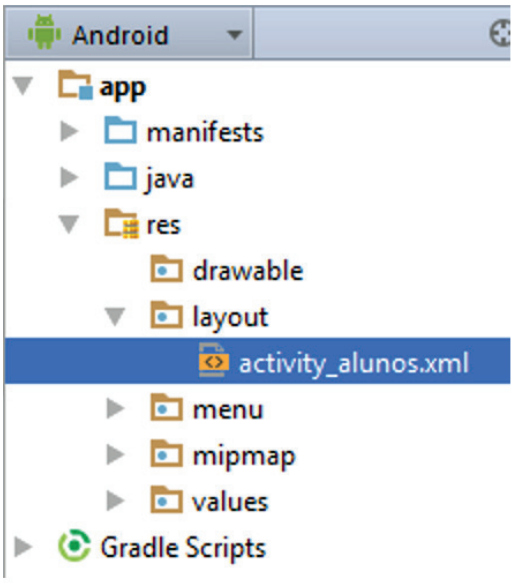


Figura 12

Dentro deste XML, preencha com as linhas abaixo:

```
<LinearLayout
 xmlns:android=”http://schemas.android.com/apk/res/android”
 android:layout_width=”wrap_content”
 android:layout_height=”wrap_content”>

 <ListView
 android:id=”@+id/alunos”
 android:layout_width=”wrap_content”
 android:layout_height=”wrap_content”>
 </ListView>

</LinearLayout>
```

No módulo "Design", o mesmo ficará da seguinte forma:

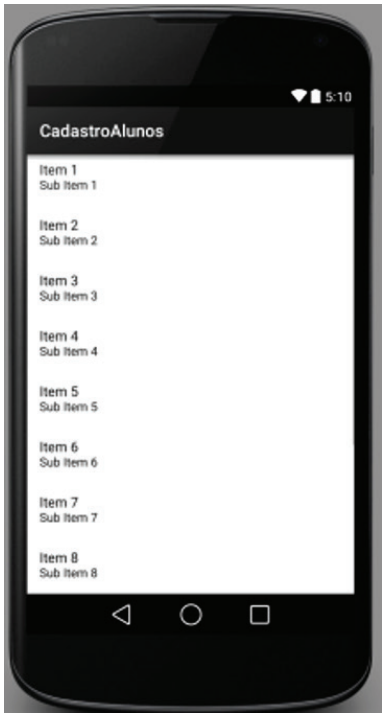


Figura 13

Para demonstrar a funcionalidade da lista, iremos acrescentar alguns nomes fixos que serão acrescentados um objeto do tipo ArrayAdapter. Iremos precisar, também, de um objeto do tipo ListAdapter, onde será a representação vertical de uma lista já preparada para ser colocada em uma tela do Android. E para agregar esse item à tela de aplicação, precisamos utilizar o objeto ListView. Esse objeto já está pronto e disponível, basta utilizá-lo usando a seguinte instrução:

(ListView) findViewById(R.id.alunos);

Note que, dentro do java, é possível encontrar um objeto de visualização pelo id. A maneira como é realizado esse link no XML é através da tag @+id/alunos.

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <ListView
        android:id="@+id/alunos"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
    </ListView>

</LinearLayout>
```

Figura 14

Modifique o código da classe **Alunos.java** pelo código abaixo para que possamos criar a lista estática de alunos.

Primeiramente, importe os objetos que iremos utilizar:

```
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import java.util.Arrays;
import java.util.List;
```

No método onCreate, subrescreva o código abaixo, onde iremos criar uma lista fixa de nomes:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 List <String> alunos = Arrays.asList("Joao da Silva", "Maria
dos Santos");
 ArrayAdapter<String> adapter = new ArrayAdapter<String>
(this,android.R.layout.simple_list_item_1,alunos);
 ListView listaAlunos = (ListView) findViewById(R.id.alunos);
 listaAlunos.setAdapter(adapter);
}
}
```

Execute o aplicativo e teremos a tela abaixo:

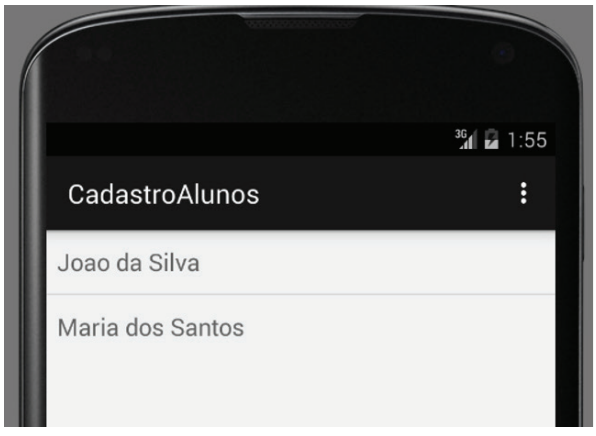


Figura 15

Alertas através do objeto Toast

Pequenos avisos podem ser exibidos através do objeto Toast (Torradeira), no qual exibe um aviso na tela por um determinado tempo. A duração pode ser configurada pelos comandos:

`Toast.LENGTH_LONG=1` e `Toast.LENGTH_SHORT=0.`

`Toast.makeText`(context,"Meu texto de aviso", `Toast.LENGTH_LONG).show();`

Quando um Toast é criado, uma mensagem será exibida na tela como um texto informativo. Diferente de um alerta comum, você pode continuar a utilizar sua aplicação normalmente enquanto o Toast é exibido. Podemos integrá-lo à nossa aplicação para que quando o usuário clicar no aluno, mostrar a posição do mesmo na lista.

Integrando o objeto Toast ao nosso projeto

Iremos incorporar o objeto Toast no evento click, através do `Listener OnItemClickListener`. Altere o método `onCreate()` da classe **Alunos.java** conforme abaixo:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 List <String> alunos = Arrays.asList("Joao da Silva", "Maria
dos Santos");
 ArrayAdapter<String> adapter = new ArrayAdapter<String>
(this,android.R.layout.simple_list_item_1,alunos);
 ListView listaAlunos = (ListView) findViewById(R.id.alunos);
 listaAlunos.setAdapter(adapter);

 listaAlunos.setOnItemClickListener(new AdapterView.OnItem-
ClickListener() {
 public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
  Toast.makeText(Alunos.this, "Posição selecionada: " + posi-
cao, Toast.LENGTH_LONG).show();
 }
 });
}
```

Execute e veja o resultado, selecionando o primeiro item da lista "João da Silva". Lembrando que a primeira posição é representado pelo id 0. Podemos também tratar outros eventos.

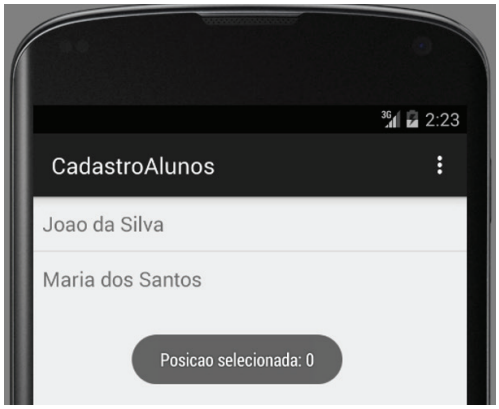


Figura 16

Abaixo uma declaração para um click e para o longClick:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 List <String> alunos = Arrays.asList("Joao da Silva", "Maria
dos Santos");
 ArrayAdapter<String> adapter = new ArrayAdapter<String>
(this,android.R.layout.simple_list_item_1,alunos);
 ListView listaAlunos = (ListView) findViewById(R.id.alunos);
 listaAlunos.setAdapter(adapter);

 listaAlunos.setOnItemClickListener(new AdapterView.OnItem-
ClickListener() {
 public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
  Toast.makeText(Alunos.this, "Posicao selecionada: " + posi-
cao, Toast.LENGTH_LONG).show();
 }
 });

 listaAlunos.setOnItemLongClickListener(new AdapterView.
OnItemLongClickListener() {
 public boolean onItemLongClick(AdapterView<?> adapter, View
view, int posicao, long id) {
  Toast.makeText(Alunos.this, "Posicao selecionada com toque
mais longo: " + posicao, Toast.LENGTH_LONG).show();
  return false;
 }
 });

}
```

Execute novamente e clique por um período mais longo. Neste caso, foi selecionado o click longo na Maria dos Santos.



Figura 17

O próximo passo é criarmos um menu para que possamos navegar entre as telas. Vamos utilizar o que aprendemos anteriormente com menus.

Dentro do método onCreateOptionsMenu, iremos acrescentar os elementos abaixo:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
 boolean resultado = super.onCreateOptionsMenu(menu);
 menu.add(0,0,0,"Novo");
 menu.add(0,1,0,"Sincronizar");
 menu.add(0,2,0,"Galeria");
 menu.add(0,3,0,"Mapa");
 return resultado;
}
```

Dos três argumentos inteiros informado no método add, o primeiro é o grupo do menu (uma divisão visualmente indiferente, mas que possibilita, por exemplo, desabilitar todo um grupo de menu), o segundo é o ID (utilizado para comparar com o item que foi clicado) e o terceiro define a ordem que o menu deve aparecer (0 deixa a cargo do Android).

Como comportamento padrão, o sistema colocará como nome do botão, o parâmetro passado no método add, mas, se preferir, pode-se adicionar um ícone. Basta chamar no ítem de menu recém-criado, o método seticon, passando um R.drawable.nomeDaImagem.

Exemplo:

Menuitem mapa = menu.add(0,3,0,"Mapa);

mapa.setIcon(R.drawabble.mapa);



Figura 18

Há, também, o método onOptionsItemSelected(MenuItem item) na Activity, que deve ser sobrescrito para saber se determinado item do menu foi selecionado:

```
@Override
@Override
public boolean onOptionsItemSelected(MenuItem item) {
 if (item.getItemId() == 0) {
 Toast.makeText(Alunos.this,"Item Novo Aluno seleciona-
do",Toast.LENGTH_LONG).show();
 }
 return false;
}
```

Deve-se retornar false para que esse evento continue a ser processado pelo Android.

Context menu

Outra prática bastante usada é o menu de contexto. Qualquer componente (View) pode ter um menu de contexto vinculado a ele. Para ativar o menu de nossa listaAlunos, basta invocarmos registerForContextMenu(listaAlunos). Assim, o menu contextual aparecerá sempre que o usuário der um **clique longo em um item da lista**. Depois de invocado, o Android disparará uma chamada para onCreateContextMenu.

Para isso, iremos criar o método abaixo:

```
public void onCreateContextMenu(ContextMenu menu, View v,
ContextMenu.ContextMenuInfo menuInfo){
 super.onCreateContextMenu(menu, v, menuInfo);
 menu.add(0,0,0,"Ligar");
 menu.add(0,1,0,"Enviar SMS");
 menu.add(0,2,0,"Achar no Mapa");
 menu.add(0,3,0,"Navegar no site");
 menu.add(0,4,0,"Deletar");
 menu.add(0,5,0,"Enviar e-mail");
}
```

Neste caso, não estamos considerando qual View está sendo usada, já que apenas invocaremos registerForContextMenu para a listaAlunos no método onCreate.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 List <String> alunos = Arrays.asList("Joao da Silva", "Maria
dos Santos");
 ArrayAdapter<String> adapter = new ArrayAdapter<String>
(this,android.R.layout.simple_list_item_1,alunos);
 ListView listaAlunos = (ListView) findViewById(R.id.alunos);
 listaAlunos.setAdapter(adapter);

 listaAlunos.setOnItemClickListener(new AdapterView.OnItem-
ClickListener() {
 public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
 Toast.makeText(Alunos.this, "Posicao selecionada: " + posi-
cao, Toast.LENGTH_LONG).show();
 }
 });

 listaAlunos.setOnItemLongClickListener(new AdapterView.
OnItemLongClickListener() {
 public boolean onItemLongClick(AdapterView<?> adapter, View
view, int posicao, long id) {
```

```
    Toast.makeText(Alunos.this, "Posicao selecionada com toque
mais longo: " + posicao, Toast.LENGTH_LONG).show();
    return false;
  }
  });

registerForContextMenu(listaAlunos);
}
```

Neste estágio, ainda não teremos resultado, visto que teremos que criar nosso banco de dados ainda.

Cadastro de Alunos

Não utilizaremos somente Toast´s, precisaremos realizar um formulário para cadastro, e para isso devemos chamar outra Activity. Dessa forma, será possível navegar pelas telas da nossa aplicação por meio dos menus e botões.

O processo é simples: criaremos uma nova Activity, que controlará as ações da nova tela, e um novo Android XML para criar um novo layout de tela para o cadastro dos alunos.

Em seguida, bastará fazer com que o clique em determinado menu, chame o método startActivity(...), passando para ele um Intent: uma intenção de ir para outra Activity. O intent, por sua vez, precisará receber o objeto de contexto atual e a classe que controla a tela para onde queremos ir.

```
Intent intent = new Intent(ContextoAtual.this, NovaActivity.class);

startActivity(intent);
```

Na nova acitivy, dentro do onCreate, utilizamos o método setContentView para renderizar a nova tela, recuperando as informações do XML:

```
setContentView(R.layout.novaActivity);
```

Ícone da sua aplicação

Para colocar um ícone no seu aplicativo, você terá que adicionar um arquivo de imagem na pasta drawable da aplicação e declará-la no seu AndroidManifest.xml.

Na tab Application, deve ser adicionada a informação do local do logo:
`<application android:icon=”@drawable/icon”>`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.usuario.cadastroalunos" >

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="CadastroAlunos"
        android:theme="@style/AppTheme">
        <activity
            android:name=".Alunos"
            android:label="CadastroAlunos" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Figura 19

Formulário para inclusão dos alunos

Agora, iremos criar o formulário para cadastro de alunos, no qual terá no nome de formulário.xml.

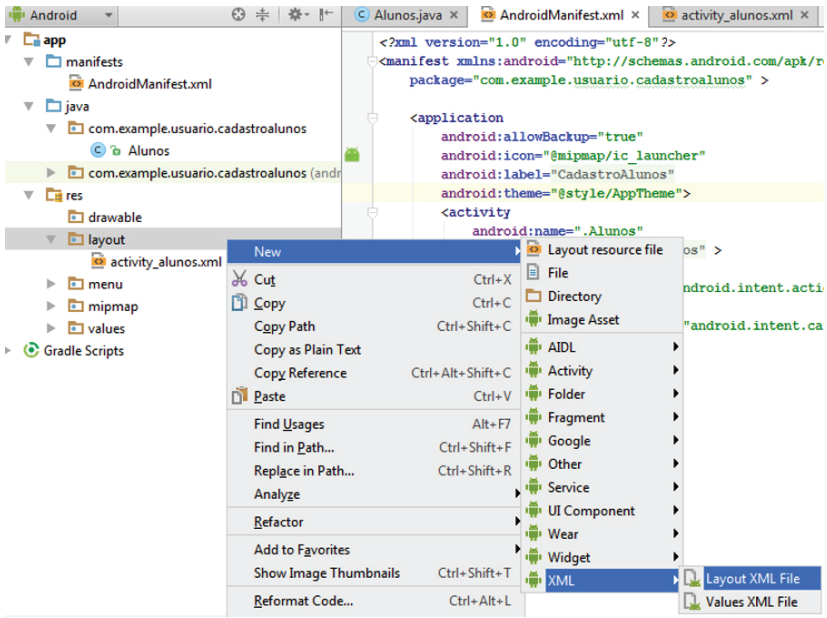


Figura 20

Substitua o código gerado pelo layout abaixo para que o mesmo fique com a seguinte aparência:

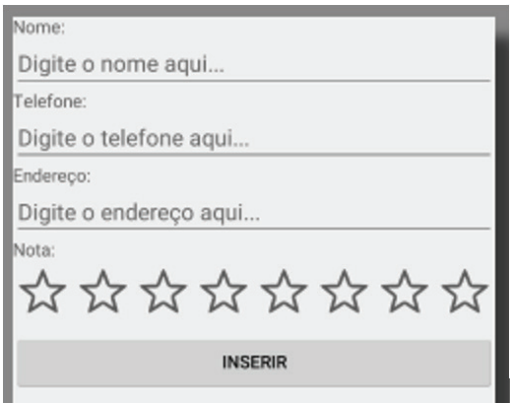


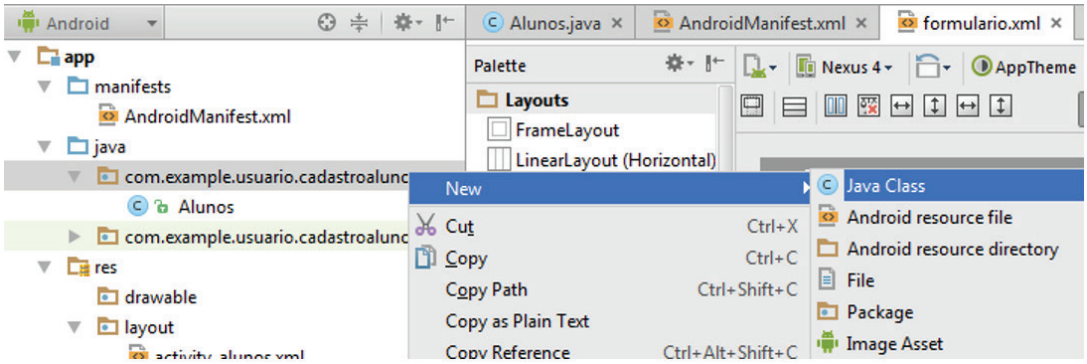
Figura 21

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/
res/android"
 android:layout_width="fill_parent"
 android:layout_height="fill_parent">
 <LinearLayout
 android:layout_width="fill_parent"
 android:layout_height="fill_parent"
 android:orientation="vertical">
 <TextView
  android:text="Nome:"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  />
 <EditText
  android:id="@+id/nome"
  android:hint="Digite o nome aqui..."
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  />
 <TextView
  android:text="Telefone:"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  />
 <EditText
  android:id="@+id/telefone"
  android:hint="Digite o telefone aqui..."
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  />
<TextView
  android:text="Endereço:"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  />
```

```
    <EditText
     android:id="@+id/endereco"
     android:hint="Digite o endereço aqui..."
     android:layout_width="fill_parent"
     android:layout_height="wrap_content"
     />
    <TextView
     android:text="Nota:"
     android:layout_width="fill_parent"
     android:layout_height="wrap_content"
     />
    <RatingBar
     android:id="@+id/nota"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:numStars="10"
     />
   <Button
     android:id="@+id/botao"
     android:layout_width="fill_parent"
     android:layout_height="wrap_content"
     android:text="Inserir"
     />
    </LinearLayout>
   </ScrollView>
```

Observação: o componente RatingBar permite dar um valor float de maneira rápida e amigável, sem a necessidade de um teclado. Para realizarmos a inclusão, iremos criar uma classe de modelo. Neste caso, será criada uma classe que representará a entidade Aluno com alguns atributos para guardar as informações.

Adicione uma nova classe com nome Aluno no pacote **cadastroalunos**.



Os atributos que iremos utilizar são:

```
package com.example.usuario.cadastroalunos;

public class Aluno {
 private Long id;
 private String nome;
 private String telefone;
 private String endereco;
 private double nota;
}
```

Para não ter que criar todos os getters e setters manualmente, você poderá clicar na classe com o botão direito do mouse e selecionar o item "Refactor" o item "Encapsulate Fields", conforme tela abaixo.

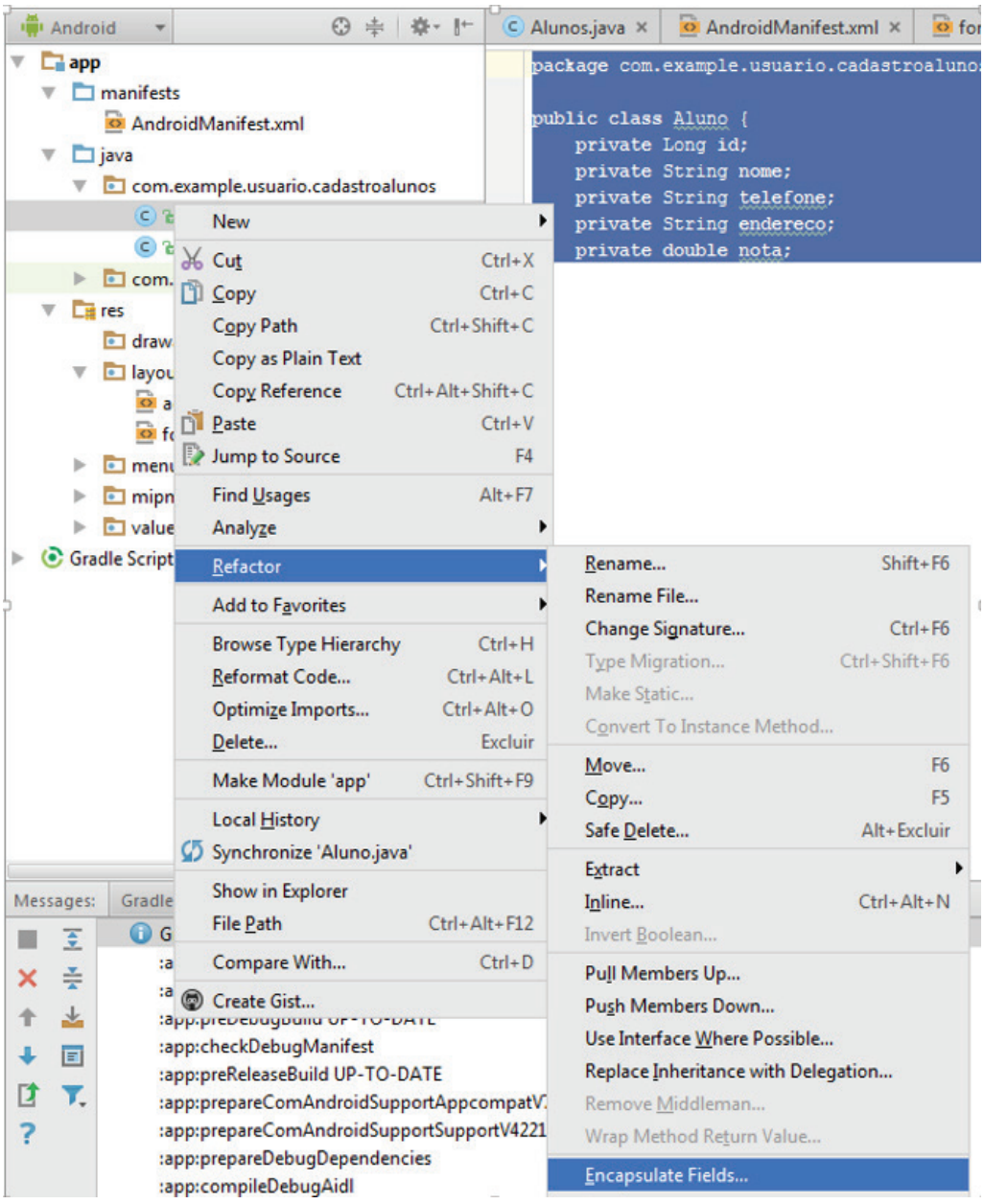


Figura 23

Selecione todos os atributos:

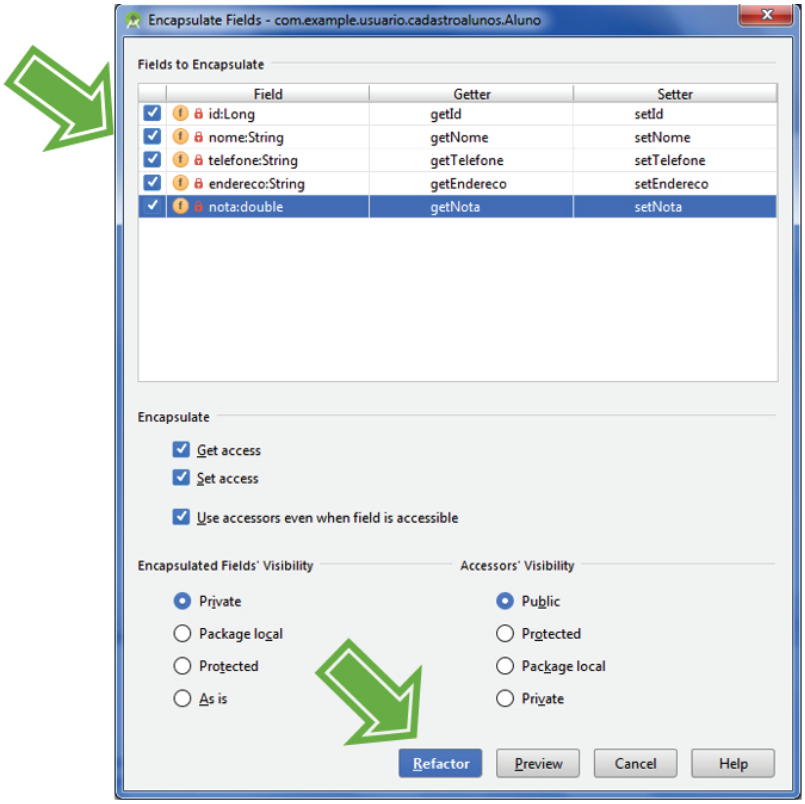


Figura 24

E sua classe ficará assim:

```
package com.example.usuario.cadastroalunos;

public class Aluno {
 private Long id;
 private String nome;
 private String telefone;
 private String endereco;
 private double nota;

 public Long getId() {
 return id;
 }
 public void setId(Long id) {
 this.id = id;
 }

 public String getNome() {
 return nome;
 }

 public void setNome(String nome) {
 this.nome = nome;
```

```
    }

     public String getTelefone() {
     return telefone;
     }

     public void setTelefone(String telefone) {
     this.telefone = telefone;
     }

     public String getEndereco() {
     return endereco;
     }

     public void setEndereco(String endereco) {
     this.endereco = endereco;
     }

     public double getNota() {
     return nota;
     }

     public void setNota(double nota) {
     this.nota = nota;
     }
    }
```

Objeto DAO

DAO (Data Accesss Object) é um pattern que deve funcionar como um tradutor entre o mundo relacional e o mundo Orientado a Objetos. No caso do Android, o banco de dados será o SQLite, um banco mais leve, que salva os objetos em arquivos locais. Para trabalhar com ele, basta seguir a abordagem padrão: criar uma classe chamada **AlunoDAO.java.** A diferença será que ela deve estender SQLLiteOpenHelper e ter um construtor que recebe um objeto do tipo Context. Esse objeto traz informações sobre a aplicação.

Precisamos receber esse objeto porque, para construir corretamente o DAO, é necessário chamar o construtor da classe mãe, passando o contexto e algumas configurações do seu banco em particular. Logo, teremos dois métodos:

- onCreate: executado na criação do banco;
- onUpgrade: executado na alteração da estrutura do banco.

O onCreate vai ser invocado sempre que a tabela não existir, por exemplo, no primeiro acesso à aplicação. Já o método onUpgrade é mais específico. Sempre que quisermos atualizar a estrutura da nossa tabela, este método será chamado para executar o SQL necessário para realizar a atualização. No nosso caso, sempre apagaremos a tabela e criaremos uma nova.

Tipos de dados suportados pelo SQLite

Os tipos de dados suportados pelo SQLite são: INTEGER, REAL, TEXT, BLOB, NULL mas são permitidos alguns sinônimos, por exemplo:

```
private static final String TABLE_CREATE =

 "CREATE TABLE contatos (nome String PRIMARY KEY, "

                    + " celular String, email String, salario float)";
```

Para podermos invocar métodos para executar operações no banco como insert, update e/ou delete, usaremos objeto do tipo SQLietDatabase, recebido através do método getWritableDatabase que herdamos de SQLiteOpenHelper, e também recebemos no onCreate. Há, também, o método getReadableDatabase, que também devolve uma referência para SQLiteDatabase, consumindo menos recursos, porém, possibilitando apenas leitura.

Para criarmos o banco, precisaremos executar uma query SQL, indicando as colunas da nossa tabela e seus respectivos tipos de dados. Portanto, iremos criar uma tabela com a mesma estrutura da nossa classe.

Essa query deve ser executada na criação do banco de dados, portanto no método onCreate e ela preparará nossa infraestrutura para armazenar os alunos. O SQLiteDatabase tem o método execSQL para isso.

Crie outra classe com o nome **AlunoDAO.java** no mesmo pacote e iremos estender a classe SQLiteOpenHelper. Iremos implementar os métodos onCreate, onUpgrade e inserir.

```
package com.example.usuario.cadastroalunos;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import java.util.ArrayList;
import java.util.List;


public class AlunoDAO extends SQLiteOpenHelper {
 private static final int VERSAO = 1;
 private static final String TABELA = “Aluno”;
 private static final String DATABASE = “Banco”;

 public AlunoDAO(Context context){
 super(context,DATABASE, null, VERSAO);
}
 public void onCreate(SQLiteDatabase db){
 String ddl = “CREATE TABLE “ + TABELA +
   “ (id INTEGER PRIMARY KEY, “ +
   “ nome TEXT UNIQUE NOT NULL, “ +
   “ telefone TEXT, endereco TEXT, “ +
   “ nota REAL);”;
 db.execSQL(ddl);
 }
ublic void onUpgrade(SQLiteDatabase db, int oldVersion, int
newVersion){
 String sql = “DROP TABLE IF EXISTS “ + TABELA;
 db.execSQL(sql);
 onCreate(db);
 }
 public void inserir(Aluno aluno){
 ContentValues values = new ContentValues();
 values.put(“nome”, aluno.getNome());
 values.put(“telefone”,aluno.getTelefone());
 values.put(“endereco”, aluno.getEndereco());
 values.put(“nota”, aluno.getNota());
 getWritableDatabase().insert(TABELA,null, values);
 }
}
```

Devemos tomar cuidado com a implementação do método onUpgrade, onde, neste caso, está apagando a tabela, mas será útil somente para nossos testes.

```
private static final String[] COLS = {“id,”,”nome”,”telefone”,”endereco”,”nota”};
```

Para que sejam exibidas as informações dos registros cadastrados no banco de dados, será preciso saber o nome da tabela, e quais colunas gostaríamos de recuperar. Pensando nisso, adicionaremos outra constante à classe **AlunoDAO.java**, onde conterá um array de Strings que representa as colunas a serem recuperadas. Adicione, logo abaixo, da definição da classe:

```
public class AlunoDAO extends SQLiteOpenHelper {
 private static final String[] COLS = {“id”,”nome”,”telefone”,
”endereco”,”nota”};
```

Agora, iremos abrir um cursor e completar o método, criando uma lista de alunos. Note que a ordem do array criado acima é muito importante, pois ela será o retorno do seu Cursor.

```
public List<Aluno> getLista(){
 List<Aluno> alunos = new ArrayList<Aluno>();
 Cursor c = getWritableDatabase().query(TABELA, COLS, null,
null, null, null, null);
 while (c.moveToNext()){
 Aluno aluno = new Aluno();
 aluno.setId(c.getLong(0));
 aluno.setNome(c.getString(1));
 aluno.setTelefone(c.getString(2));
 aluno.setEndereco(c.getString(3));
 aluno.setNota(c.getDouble(4));
 alunos.add(aluno);
 }
 c.close();
 return alunos;
}
```

O método query, recebe a tabela e um conjunto de colunas a serem recuperadas. Outros parâmetros que estão nulos permitem, também, fazer select, group By, having e Order by. Explore a API para aprender mais sobre essas opções.

Alguns métodos do objeto android.database.sqlite.SQLiteDatabase

- **public void execSQL(String sql)**
  Executa um comando SQL. Este método não poderá executar instruções SQL que possuam retorno de valores. Utilizar este método para executar comandos SQL que modifiquem o BD.

- **public Cursor query(String table, String[] columns, String selection, String[] selectionArgs, String groupBy, String having, String orderBy)**Método utilizado para recuperar dados de uma tabela e retornar um grupo de resultados do tipo Cursor. O comando SQL será construído com os argumentos especificados (nome da tabela, campos, filtros, ordenação etc.).
- **public Cursor rawQuery(String sql, String[] selectionArgs)** Também utilizado para recuperar dados de uma tabela e retornar um grupo de resultados do tipo Cursor. No comando de consulta em sql, podemos incluir ?, cujos valores especificaremos em selectionArgs (ou colocar null neste segundo parâmetro).

Agora, ajustaremos a nossa lista (alunos), para que sejam substituídos os nomes fixos que foram atribuídos anteriormente pelo recuperados do banco. Na classe **Alunos.java.**, procure o método onCreate.

Perceba que declaramos os atributos alunos e listaAlunos como private na classe.

```
package com.example.usuario.cadastroalunos;

import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.ContextMenu;
import android.view.Menu;
import android.view.MenuItem;

import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import java.util.Arrays;
import java.util.List;

public class Alunos extends ActionBarActivity {
 private List<Aluno> alunos;
 private ListView listaAlunos;

@Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 listaAlunos = (ListView) findViewById(R.id.alunos);
 AlunoDAO dao = new AlunoDAO(this);
 alunos = dao.getLista();
 dao.close();

ArrayAdapter<Aluno> adapter = new ArrayAdapter<Aluno>
(this,android.R.layout.simple_list_item_1,alunos);

listaAlunos.setAdapter(adapter);
```

```
listaAlunos.setOnItemClickListener(new AdapterView.OnItemCli-
ckListener() {
  public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
  Toast.makeText(Alunos.this, "Posicao selecionada: " + posi-
cao, Toast.LENGTH_LONG).show();
  }
 });

listaAlunos.setOnItemLongClickListener(new AdapterView.OnI-
temLongClickListener() {
  public boolean onItemLongClick(AdapterView<?> adapter, View
view, int posicao, long id) {
  Toast.makeText(Alunos.this, "Posicao selecionada com toque
mais longo: " + posicao, Toast.LENGTH_LONG).show();
  return false;
  }
 });

 registerForContextMenu(listaAlunos);
 }

@Override
 public boolean onCreateOptionsMenu(Menu menu) {
 boolean resultado = super.onCreateOptionsMenu(menu);
 menu.add(0,0,0,"Novo");
 menu.add(0,1,0,"Sincronizar");
 menu.add(0,2,0,"Galeria");
 menu.add(0,3,0,"Mapa");
 return resultado;
 }

@Override
 public boolean onOptionsItemSelected(MenuItem item) {
 if (item.getItemId() == 0) {
  Toast.makeText(Alunos.this,"Item Novo Aluno selecionado",-
Toast.LENGTH_LONG).show();
 }
 return false;
 }

public void onCreateContextMenu(ContextMenu menu, View v, Con-
textMenu.ContextMenuInfo menuInfo){
 super.onCreateContextMenu(menu, v, menuInfo);
 menu.add(0,0,0, "Ligar");
 menu.add(0,1,0,"Enviar SMS");
 menu.add(0,2,0,"Achar no Mapa");
 menu.add(0,3,0,"Navegar no site");
 menu.add(0,4,0,"Deletar");
 menu.add(0,5,0,"Enviar e-mail");
 }
}
```

No Formulário, iremos acrescentar o código abaixo para o botão Inserir. Iremos recuperar os dados da tela e preencher um objeto Aluno e passar para o AlunoDAO para que salve.

Crie uma nova classe com o nome Formulário. No método onCreate do formulário, precisamos adicionar um listener para o botão "Inserir" para capturar os dados do aluno digitado dentro do formulário. Após recuperar esses dados, gravaremos no DAO.

Veja código da classe **Formulario.java:**

```
package com.example.usuario.cadastroalunos;

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.RatingBar;

public class Formulario extends Activity {
 private Aluno aluno = new Aluno();
 private EditText nome;
 private EditText telefone;
 private EditText endereco;
 private RatingBar nota;

 public void onCreate (Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.formulario);
 this.buscaComponentesDaTela();

Button botao = (Button) findViewById(R.id.botao);
 botao.setOnClickListener(new View.OnClickListener(){
  public void onClick(View v){
  populaAlunoComDadosDaTela();
  AlunoDAO dao = new AlunoDAO(Formulario.this);
  dao.inserir(aluno);
  dao.close();
  finish();
  }
 });
 }

 private void buscaComponentesDaTela() {
 this.nome = (EditText) findViewById(R.id.nome);
 this.telefone = (EditText) findViewById(R.id.telefone);
 this.endereco = (EditText) findViewById(R.id.endereco);
 this.nota = (RatingBar) findViewById(R.id.nota);
 }
```

```
    private void populaAlunoComDadosDaTela() {
     this.aluno.setNome(this.nome.getEditableText().toString());
     this.aluno.setTelefone(this.telefone.getEditableText().to-
    String());
     this.aluno.setEndereco(this.endereco.getEditableText().to-
    String());
     this.aluno.setNota(this.nota.getRating());
     }
    }
```

Agora, precisamos alterar o clique do menu "Novo" para chamar o Formulário criado. Basta chamar a Activity do formulário através do método startActivity que recebe um Intent.

Altere o método onOptionsItemSelected do arquivo **Alunos.java** para que, ao invés de mostrar um Toast, passe a chamar a activity Formulário e importe ele para a classe:

```
    import android.support.v7.app.ActionBarActivity;
    import android.os.Bundle;
    import android.view.ContextMenu;
    import android.view.Menu;
    import android.view.MenuItem;

    import android.view.View;
    import android.widget.AdapterView;
    import android.widget.ArrayAdapter;
    import android.widget.ListView;
    import android.widget.Toast;
    import java.util.Arrays;
    import java.util.List;
    import android.content.Intent;
```

Método:

```
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
     if (item.getItemId() == 0) {
     Intent intent = new Intent(Alunos.this, Formulario.class);
     startActivity(intent);
     return false;
     }else{
     return super.onOptionsItemSelected(item);
     }
    }
```

É necessário a configuração no arquivo AndroidManifest.xml para mapear o formulário novo.

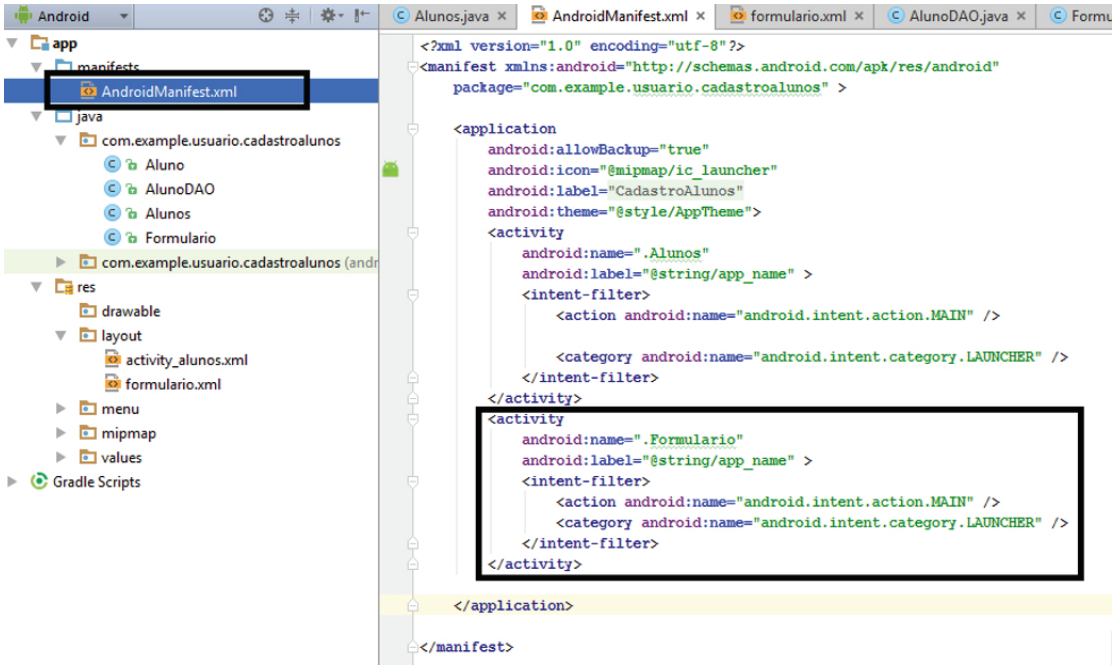


Figura 25

Portanto, dentro do arquivo `AndroidManifest.xml`, dentro da `tag <application,` acrescente o seguinte código:

```
<activity
 android:name=".Formulario"
 android:label="@string/app_name" >
 <intent-filter>
 <action android:name="android.intent.action.MAIN" />
 <category android:name="android.intent.category.LAUNCHER" />
 </intent-filter>
</activity>
```

Execute a aplicação, e já estaremos inserindo alunos no banco.

O único problema que tivemos foi que a nossa listagem exibiu uns valores estranhos para cada usuário, conforme tela abaixo:

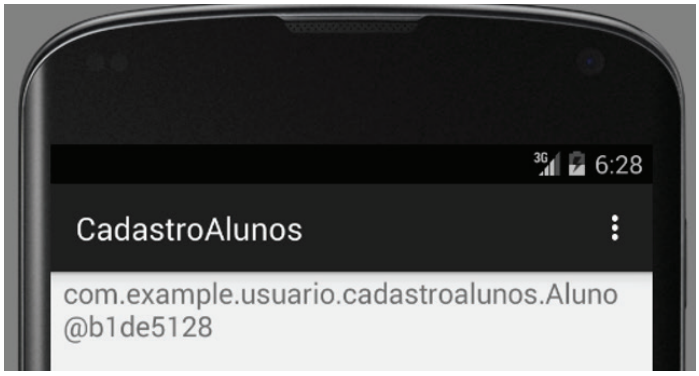


Figura 26

O detalhe é que, para cada Aluno dentro do ArrayAdapter, é chamado o método toString, e como não sobrescrevemos ele na nossa classe, está sendo exibido o toString padrão de Object. Para resolvermos isso, iremos acrescentar o método toString na classe **Aluno.java.**

```
public String toString(){
 return id + " - " + nome;
}
```

Quando chamamos o método finish, ele finaliza a Activity que está em primeiro plano e, ao voltar na Activity anterior, ele executa o método onResume() da classe **Alunos.java.**

```
protected void onResume(){
 super.onResume();
 carregaLista();
}
```

Para melhoramos ainda mais o nosso sistema, vamos criar o método carregaLista na classe **Alunos.java** para centralizarmos o carregamento da nossa lista, e vamos chamá-lo no onResume();

```
private void carregaLista(){
 AlunoDAO dao = new AlunoDAO(this);
 List<Aluno> alunos = dao.getLista();
 dao.close();
 ArrayAdapter<Aluno> adapter = new ArrayAdapter<Aluno>
(this,android.R.layout.simple_list_item_1,alunos);
 listaAlunos.setAdapter(adapter);
}
```

Iremos refatorar o código do onCreate para invocar o carregaLista(), ao invés de deixar esse código duplicado:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_alunos);

 listaAlunos = (ListView) findViewById(R.id.alunos);

 carregaLista();

listaAlunos.setOnItemClickListener(new AdapterView.OnItem-
ClickListener() {
 public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
  Toast.makeText(Alunos.this, "Posicao selecionada: " + posi-
cao, Toast.LENGTH_LONG).show();
 }
 });
```

```
    listaAlunos.setOnItemLongClickListener(new AdapterView.OnItem-
    LongClickListener() {
     public boolean onItemLongClick(AdapterView<?> adapter, View
    view, int posicao, long id) {
      Toast.makeText(Alunos.this, "Posicao selecionada com toque
    mais longo: " + posicao, Toast.LENGTH_LONG).show();
      return false;
     }
     });

     registerForContextMenu(listaAlunos);
    }
```

Apagando um registro:

Para remover aluno, basta chamar o método delete, passando a TABELA, e a query de busca na classe **Alunos.java**. Primeiro, iremos criar uma variável do tipo Aluno, no qual armazenará o aluno que será selecionado pelo clique longo.

```
    public class Alunos extends ActionBarActivity {
     private List<Aluno> alunos;
     private ListView listaAlunos;
     private Aluno alunoSelecionado;
```

Iremos substituir o código do clique longo, para que seja passado o id do Aluno selecionado:

```
    listaAlunos.setOnItemLongClickListener(new AdapterView.OnItemLong-
    ClickListener() {
     public boolean onItemLongClick(AdapterView<?> adapter, View view,
    int posicao, long id) {
     alunoSelecionado = (Aluno) adapter.getItemAtPosition(posicao);
     registerForContextMenu(listaAlunos);
     return false;
     }
    });
```

Crie um método deletar na classe **AlunoDAO.java** para possibilitar o sistema excluir alunos:

```
    public void deletar(Aluno aluno){
     getWritableDatabase().delete(TABELA, "id=?", new String[]
    {aluno.getId().toString()});
    }
```

Agora, precisamos fazer com que o evento disparado pela seleção do menu "Deletar" invoque a exclusão do aluno; para isso, iremos alterar o método onCreateContextMenu da classe **Alunos.java**, adicionando o evento click no objeto MenuItem, conforme código abaixo:

```
public void onCreateContextMenu(ContextMenu menu, View v,
ContextMenu.ContextMenuInfo menuInfo){
 super.onCreateContextMenu(menu, v, menuInfo);
 menu.add(0,0,0,"Ligar");
 menu.add(0,1,0,"Enviar SMS");
 menu.add(0,2,0,"Achar no Mapa");
 menu.add(0,3,0,"Navegar no site");
 menu.add(0,5,0,"Enviar e-mail");

 MenuItem deletar = menu.add(0,4,0,"Deletar");

 deletar.setOnMenuItemClickListener(new MenuItem.OnMenuItem-
ClickListener() {
 public boolean onMenuItemClick(MenuItem item) {
  AlunoDAO dao = new AlunoDAO(Alunos.this);
  dao.deletar(alunoSelecionado);
  dao.close();

  carregaLista();
  return false;
 }
 });
}
```

Caso queira adicionar uma janela de **Alerta** antes da exclusão, você poderá implementar no evento clique o código abaixo.

```
deletar.setOnMenuItemClickListener(new MenuItem.OnMenuItem-
ClickListener() {
 public boolean onMenuItemClick(MenuItem item) {
 new AlertDialog.Builder(Alunos.this)
  .setIcon(android.R.drawable.ic_dialog_alert)
  .setTitle("Exclusao de Aluno")
  .setMessage("Deseja realmente excluir o aluno selecionado?")
  .setPositiveButton("Sim", new DialogInterface.OnClickListen-
er() {
   public void onClick(DialogInterface dialog, int which){
   AlunoDAO dao = new AlunoDAO(Alunos.this);
   dao.deletar(alunoSelecionado);
   dao.close();
   carregaLista();
   }
  }).setNegativeButton("Nao",null).show();
 return false;
 }
});
```



Deverão ser importados os objetos AlertDialog e DialogInterface na classe Alunos.java.

```
import android.app.AlertDialog;
import android.content.DialogInterface;
```

Atualizando um registro:

Poderemos aproveitar a tela de inserção criada anteriormente e atribuir ao evento Click do listener para editar esse registro. Só que devemos passar dados de uma Activity para a outra e iremos utilizar o atributo putExtra, no qual adiciona elementos como um mapa e possui sobrecargas para diversos tipos como int, Sring e Serializable.

Iremos alterar o método clique na classe **Alunos.java,** passando o aluno selecionado como parâmetro extra:

```
listaAlunos.setOnItemClickListener(new AdapterView.OnItem-
ClickListener() {
 public void onItemClick(AdapterView<?> adapter, View view,
int posicao, long id) {
 Intent edicao = new Intent(Alunos.this, Formulario.class);
 edicao.putExtra("alunoSelecionado", (Aluno) listaAlunos.
getItemAtPosition(posicao));
 startActivity(edicao);

 }
});
```

Como iremos transportar um objeto entre as Activitys, devemos implementar Serialização na classe aluno, portanto, altere a classe Aluno.java conforme abaixo:

```
import java.io.Serializable;

public class Aluno implements Serializable {
```

No formulário (**Formulario.java**), quando a activity for criada, precisamos descobrir se um aluno está sendo editado, ou será apenas uma inserção. Para isso, dentro do método onCreate, precisaremos consistir conforme código abaixo:

```
public void onCreate (Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.formulario);
 this.buscaComponentesDaTela();

 aluno = (Aluno) getIntent().getSerializableExtra("alunoSelec
ionado");
 Button botao = (Button) findViewById(R.id.botao);

 if (aluno == null) {
 aluno = new Aluno();
 }else{
 botao.setText("Alterar");
 populaTelaComDadosDoAluno();
 }
 botao.setOnClickListener(new View.OnClickListener(){
 public void onClick(View v){
  populaAlunoComDadosDaTela();
  AlunoDAO dao = new AlunoDAO(Formulario.this);
  dao.inserir(aluno);
  dao.close();
  finish();
 }
 });
}
```

Altere a declaração do objeto aluno logo abaixo da definição da classe para que não se instancie o objeto, mas somente declarado.

```
public class Formulario extends Activity {
 private Aluno aluno;
```

O próximo passo é criar o método populaAlunoComDadosDaTela, referenciado acima. Crie o método abaixo na classe **Formulario.java.**

```
private void populaTelaComDadosDoAluno(){
 this.nome.setText(aluno.getNome());
 this.telefone.setText(aluno.getTelefone());
 this.endereco.setText(aluno.getEndereco());
 this.nota.setRating(new Float(aluno.getNota()));
}
```

Agora, precisamos criar um método alterar na classe DAO (**AlunoDAO.java**), no qual irá alterar o aluno pelo seu id.

```
public void alterar(Aluno aluno){
 ContentValues values = new ContentValues();
 values.put("nome",aluno.getNome());
 values.put("telefone",aluno.getTelefone());
 values.put("endereco",aluno.getEndereco());
 values.put("nota",aluno.getNota());
 getWritableDatabase().update(TABELA,values, "id=?", new
String[] {aluno.getId().toString()});
}
```

E quando o botão do formulário for clicado, iremos verificar se é uma inclusão ou alteração. Para isso, no método onCreate da classe **Formulario.java**, faça a consistência abaixo no evento Click.

```
botao.setOnClickListener(new View.OnClickListener(){
 public void onClick(View v){
 populaAlunoComDadosDaTela();
 AlunoDAO dao = new AlunoDAO(Formulario.this);
 if (aluno.getId() != null){
  dao.alterar(aluno);
 }else{
  dao.inserir(aluno);
 }
 dao.close();
 finish();
 }
});
```

Abaixo, o evento onCreate completo da classe **Formulario.java:**

```
public void onCreate (Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.formulario);
 this.buscaComponentesDaTela();

 aluno = (Aluno) getIntent().getSerializableExtra("alunoSeleciona
do");
 Button botao = (Button) findViewById(R.id.botao);

 if (aluno == null) {
 aluno = new Aluno();
 }else{
 botao.setText("Alterar");
 populaTelaComDadosDoAluno();
 }
 botao.setOnClickListener(new View.OnClickListener(){
 public void onClick(View v){
  populaAlunoComDadosDaTela();
  AlunoDAO dao = new AlunoDAO(Formulario.this);
  if (aluno.getId() != null){
  dao.alterar(aluno);
  }else{
  dao.inserir(aluno);
  }
  dao.close();
  finish();
 }
 });
}
```

Pronto, agora, é executar e ver o resultado.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

SQLite no Android, Entendendo e Utilizando

Para aprofundar seus estudos sobre inserção usando banco de dados.

https://youtu.be/QQ-Ir8pwxWA

Referências

ANDROID. Android Developers. Disponível em http://developer.android.com.

LECHETA, R. R. Google Android, aprenda a criar aplicações para dispositivos móveis com o Android SDK. São Paulo: Novatec, 2010.

MEDNIEKS, Z. et. al. Desenvolvimento de Aplicações Android. São Paulo: Novatec, 2009.

LEE, W. M. Introdução ao Desenvolvimento de Aplicativos para o Android. Rio de Janeiro: Editora Ciência Moderna, 2011.

SQLITE. SQLite. Site oficial em www.sqlite.org.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000


Cruzeiro do Sul
Educacional